

## BSIDES DUBLIN

# Conference Badge

---

## Hardware CTF & Badge Reference Guide

*Challenge brief, serial CLI reference, and full solution walkthrough*

|                   |                                                           |
|-------------------|-----------------------------------------------------------|
| <b>Document</b>   | Badge CTF & CLI Reference Guide                           |
| <b>Hardware</b>   | STM32L053-based capacitive-touch “harp” badge, USB-C      |
| <b>Audience</b>   | CTF participants and badge owners                         |
| <b>Challenges</b> | 4 flags: static analysis, USB, side channel, audio puzzle |
| <b>Status</b>     | Final for distribution                                    |

## Contents

|                                        |    |
|----------------------------------------|----|
| 1. Introduction                        | 3  |
| 2. Getting Started                     | 4  |
| 2.1 Hardware setup                     | 4  |
| 2.2 Connecting to the CLI              | 4  |
| 2.3 Interface features                 | 4  |
| 3. CLI Reference                       | 5  |
| 3.1 Animations                         | 5  |
| 3.2 Music & sound                      | 5  |
| 3.3 Badge information                  | 5  |
| 3.4 System                             | 5  |
| 3.5 Advanced configuration             | 6  |
| 3.6 Hidden features                    | 6  |
| 4. The Challenges                      | 8  |
| 4.1 Flag 1: Static Analysis (Easy)     | 8  |
| 4.2 Flag 2: USB Enumeration (Medium)   | 8  |
| 4.3 Flag 3: The Harp's Song (Hard)     | 8  |
| 4.4 Flag 4: Side Channel (Medium–Hard) | 8  |
| 5. Notes for macOS Users               | 9  |
| 5.1 Confirm the badge is detected      | 9  |
| 5.2 Connect to the serial CLI          | 9  |
| 5.3 Tooling for the challenges         | 9  |
| 6. Solutions                           | 11 |
| 6.1 Flag 1: Static Analysis            | 11 |
| 6.2 Flag 2: USB Enumeration            | 11 |
| 6.3 Flag 3: The Harp's Song            | 11 |
| 6.4 Flag 4: Side Channel               | 12 |
| 6.5 Flag summary                       | 13 |

# 1. Introduction

---

Welcome, and thanks for picking up the badge. This guide brings together everything you need in one place: how to connect to the badge, what its serial interface can do, the four CTF challenges hidden inside it, and a complete set of solutions for when you want to check your work.

The badge is a small piece of hardware in its own right, an STM32L053 microcontroller driving a set of capacitive-touch “strings” that behave like a tiny harp, powered and programmed over USB-C. You can treat it as a toy, a musical instrument, or a target. All three are intended.

## How to use this document

- If you just want to play with the badge, read Section 2 (Getting Started) and Section 3 (CLI Reference).
- If you are here for the CTF, read Section 4 (The Challenges) first and treat the solutions as a last resort.
- If you are stuck, Section 6 (Solutions) walks through each flag step by step.
- macOS users should also read Section 5, which covers a few platform-specific checks.

## 2. Getting Started

The badge exposes a rich serial command-line interface (CLI) over USB. Through it, you can customise animations, sound, and power settings, read diagnostics, and, of course, work on the challenges.

### 2.1 Hardware setup

1. Set the power switch on the back of the badge to the **USB** position.
2. Connect the badge to your computer with a USB Type-C cable.

#### Note — battery mode

In battery mode, the USB peripheral is disabled to save power unless a USB host is detected immediately at boot. If the badge does not appear on your machine, switch to USB power and reconnect.

### 2.2 Connecting to the CLI

When connected, the badge appears as a virtual COM port on Windows or as a `/dev/ttyACMx` device on Linux. On macOS, it appears as `/dev/tty.usbmodem*` device (see Section 5).

Use any serial terminal software, for example, PuTTY, screen, or minicom, with the following settings:

| Setting              | Value              |
|----------------------|--------------------|
| Baud rate            | Any (9600 is fine) |
| Data / parity / stop | 8 / none / 1       |
| Flow control         | None               |

**Example (Linux):**

```
putty -serial /dev/ttyACM0
```

### 2.3 Interface features

**Tab completion.** Press Tab to complete command names or arguments.

**Ghost suggestions.** As you type, the CLI may show a lighter-coloured “ghost” suggestion. Press Right Arrow or Tab to accept it.

**Interactive pickers.** Commands such as `idle`, `play`, or `speed` entered without arguments show an interactive list to choose from.

## 3. CLI Reference

The sections below list every command grouped by function. Arguments shown in square brackets are optional; angle brackets denote a required value.

### 3.1 Animations

| Command    | Usage                | Description                                         |
|------------|----------------------|-----------------------------------------------------|
| idle       | idle [name/N]        | Set the animation to play while the badge is idle.  |
| play       | play [name/N]        | Set the animation to play when a string is touched. |
| cycle      | cycle [all/off/list] | Cycle through multiple idle animations.             |
| brightness | brightness [0-100]   | Set maximum LED brightness.                         |
| speed      | speed [factor]       | Set animation speed (e.g., speed x1.5).             |

### 3.2 Music & sound

| Command | Usage           | Description                                        |
|---------|-----------------|----------------------------------------------------|
| scale   | scale [lo/hi]   | Toggle between octaves D5–D6 and D6–D7.            |
| volume  | volume [0-100]  | Set the buzzer volume.                             |
| buzzer  | buzzer [on/off] | Enable or disable the buzzer.                      |
| note    | note [0-5]      | Play a specific note (corresponds to the strings). |

### 3.3 Badge information

| Command | Usage       | Description                                        |
|---------|-------------|----------------------------------------------------|
| name    | name [text] | Set a name for your badge.                         |
| status  | status      | Show diagnostics, uptime, and touch-sensor deltas. |
| version | version     | Show firmware version info.                        |
| id      | id          | Show the unique 96-bit Device ID.                  |

### 3.4 System

| Command  | Usage      | Description                                                      |
|----------|------------|------------------------------------------------------------------|
| reset    | reset      | Restore all settings to factory defaults.                        |
| pin      | pin <code> | Entry point for the CTF challenge.                               |
| advanced | advanced   | Access deep configuration (see 3.5).                             |
| dfu      | dfu        | Reboot the badge into the USB bootloader (for firmware updates). |

### 3.5 Advanced configuration

The advanced command tweaks system-level parameters. Usage: `advanced <parameter> <value>`.

#### Power settings

| Parameter                       | Description                                        |
|---------------------------------|----------------------------------------------------|
| <code>timeout [N.N]</code>      | Seconds of inactivity before entering light sleep. |
| <code>shutdown [N]</code>       | Minutes of inactivity before deep sleep.           |
| <code>powersave [on/off]</code> | Enable or disable power-saving features.           |

#### Touch sensor tuning

| Parameter                    | Description                                                          |
|------------------------------|----------------------------------------------------------------------|
| <code>debounce [N]</code>    | Minimum delay (ms) between note triggers.                            |
| <code>maxhold [N]</code>     | Maximum time (ms) a string can be held before auto-release.          |
| <code>tdetect [1-50]</code>  | Touch-detection sensitivity % (lower is more sensitive).             |
| <code>trelease [1-50]</code> | Release-detection sensitivity %.                                     |
| <code>tmin [10-2550]</code>  | Minimum threshold for touch delta (overrides <code>tdetect</code> ). |
| <code>tdeb [1-10]</code>     | Channel-level debounce count.                                        |

#### Sound tuning

| Parameter                      | Description                                            |
|--------------------------------|--------------------------------------------------------|
| <code>tremolo [0-5]</code>     | Tremolo effect depth.                                  |
| <code>vibrato [0-5]</code>     | Vibrato effect depth.                                  |
| <code>notes [f1,f2,...]</code> | Set custom frequencies (Hz) for each of the 6 strings. |

#### Miscellaneous

| Parameter                       | Description                                                       |
|---------------------------------|-------------------------------------------------------------------|
| <code>sleepanim [on/off]</code> | Show a subtle animation when entering light sleep.                |
| <code>wakescan [0/1/2]</code>   | Touch-scan indicator while asleep (0 = off, 1 = blink, 2 = beat). |
| <code>wakeanim [on/off]</code>  | Play a wake-up animation when waking from light sleep.            |

### 3.6 Hidden features

**Evil Mode.** A secret “Rubber Ducky” HID keyboard emulator can be unlocked. Once unlocked, the `evil` command lets you upload and run DuckyScripts.

```
advanced evil on
```

```
evil set      # enter script upload mode
evil help     # show supported DuckyScript commands
evil clear    # wipe the stored script
```

### Use responsibly

Evil Mode emulates a keyboard and will type into whatever machine the badge is plugged into. Only use it on hardware you own or are authorised to test.

## 4. The Challenges

---

Four flags are hidden in the badge. Each one rewards a different skill, so the set as a whole is a tour through hardware, firmware, USB, and side-channel analysis. Every flag uses the format `flag{...}`.

| # | Challenge       | Skill               | Difficulty  |
|---|-----------------|---------------------|-------------|
| 1 | Static Analysis | Reverse engineering | Easy        |
| 2 | USB Enumeration | USB descriptors     | Medium      |
| 3 | The Harp's Song | Audio / encoding    | Hard        |
| 4 | Side Channel    | Timing analysis     | Medium–Hard |

### 4.1 Flag 1: Static Analysis (Easy)

The firmware binary contains a flag in plaintext. Extract it using standard reverse-engineering tools.

**Hint.** *What's the first thing you run on an unknown binary?*

**Tools.** `strings`, `binwalk`, a hex editor, Ghidra.

### 4.2 Flag 2: USB Enumeration (Medium)

Not everything a USB device says is visible on the serial port. The badge has more to say if you ask the right questions.

**Hint.** *Look beyond the serial console — the badge speaks USB.*

**Tools.** `lsusb -v`, a Wireshark USB capture, USB descriptor viewers.

### 4.3 Flag 3: The Harp's Song (Hard)

The harp remembers those who play its song. Find the right melody and watch closely.

**Hint.** *The `status` command knows the tune. You'll need to read light, not text.*

**Tools.** Your eyes, patience, and knowledge of a very old encoding.

### 4.4 Flag 4: Side Channel (Medium–Hard)

The badge has a diagnostic authentication system. It looks secure, but time tells all secrets.

**Hint.** *The badge is honest about how long it takes to think. Try different codes and pay attention.*

**Tools.** A serial terminal.

## 5. Notes for macOS Users

The badge works the same way on macOS as it does on Linux and Windows, but the device name and tooling differ. The checks below help you confirm the badge is detected and connect to its CLI.

### 5.1 Confirm the badge is detected

With the badge in USB mode and connected, list the serial devices. The badge shows up as a `usbmodem` device:

```
ls /dev/tty.usbmodem*
ls /dev/cu.usbmodem*
```

If nothing is listed, confirm the cable is a data cable (not charge-only), try a different port, and make sure the power switch is set to USB.

To inspect the USB device itself, use the built-in System Information tool or the command line:

```
system_profiler SPUSBDataType
# look for the badge under the USB tree, with its
# vendor / product ID and serial number
```

### 5.2 Connect to the serial CLI

macOS ships with `screen`, which is the quickest way to connect. Use the `cu.usbmodem` device:

```
screen /dev/cu.usbmodem* 9600

# to exit screen: press Ctrl-A then K, then Y
```

Prefer a friendlier client? `minicom` and `picocom` both install cleanly via Homebrew:

```
brew install minicom picocom
picocom -b 9600 /dev/cu.usbmodem*
```

### 5.3 Tooling for the challenges

The CTF tools have direct macOS equivalents. The table below maps each task to a tool you can install with Homebrew.

| Task              | macOS tool                              | Install / notes                                              |
|-------------------|-----------------------------------------|--------------------------------------------------------------|
| String extraction | <code>strings</code>                    | Built in (Xcode command-line tools).                         |
| Firmware carving  | <code>binwalk</code>                    | <code>brew install binwalk</code>                            |
| USB descriptors   | System Information / <code>lsusb</code> | Use System Information, or <code>brew install lsusb</code> . |

| Task                      | macOS tool         | Install / notes                                          |
|---------------------------|--------------------|----------------------------------------------------------|
| USB packet capture        | Wireshark          | Install Wireshark; USB capture needs the usbcap support. |
| Firmware extraction (DFU) | dfu-util           | <code>brew install dfu-util</code>                       |
| Disassembly               | Ghidra             | Runs on macOS; requires a JDK.                           |
| Pitch detection (Flag 3)  | Web pitch detector | Any browser-based tuner works.                           |

#### macOS USB capture

Capturing USB traffic on macOS requires enabling the USB capture interface in Wireshark (the XHC interfaces). You may be prompted for administrator access the first time.

## 6. Solutions

### Spoilers ahead

This section reveals every flag and the exact steps to reach it. If you are still working the challenges, stop here.

### 6.1 Flag 1: Static Analysis

The badge ships with a USB DFU bootloader, so the firmware is easy to extract with `dfu-util`. Once you have the binary, the flag is sitting in plaintext — a simple `strings` search finds it.

```
$ dfu-util -U fw.bin
$ strings fw.bin | grep flag
flag{strings_4re_fr33}
```

**Flag:** `flag{strings_4re_fr33}`

### 6.2 Flag 2: USB Enumeration

This flag is hidden in one of the badge's USB configuration descriptors. Enumerate the device verbosely, and the flag appears as a configuration string:

```
$ lsusb -v | grep flag
iConfiguration 6 flag{lu5b_v_1337}
```

**Flag:** `flag{lu5b_v_1337}`

### 6.3 Flag 3: The Harp's Song

This flag is an Easter egg hidden in the CLI under the `status` command, which reveals the song the badge wants to hear. Among the diagnostics, `status` prints the melody:

```
> status
...
Song: A B A
```

#### Note — hardware output differs

The badge's hardware output for this part of the song shows **"A B A"**, but the intended sequence is **"B D6 B"**. Use **"B D6 B"** when playing the song. The full corrected sequence and string order are given below.

Steps:

1. Read the song from the `status` output.
2. Use a pitch-detection website to work out which string plays which note (each of the six strings maps to a note).
3. Play the strings in the correct order: **1, 3, 5, 6, 5, 3**.

4. Played correctly, the badge enters an infinite loop and transmits the flag as Morse code over the LED — read the light, not the text.

**Flag: `flag{MELODY}`**

## 6.4 Flag 4: Side Channel

The pin command verifies a code and reports how long verification took — a classic timing side channel. Brute-force the PIN one digit at a time, watching for the jump in timing that signals a correct character. Each correct digit makes verification take measurably longer.

```
> pin 12
  auth: failed [37us]
> pin 1234
  auth: failed [204us]
> pin 12345
  auth: failed [37us]
>
> pin 6234
  auth: failed [207us]
> pin 7234
  auth: failed [352us]
> pin 7334
  auth: failed [503us]
> pin 7344
  auth: failed [500us]
> pin 7354
  auth: failed [505us]
> pin 7374
  auth: failed [649us]
> pin 7373
  auth: ok [801us]
  flag{t1m1ng_4ttack}
  evil harp mode unlocked!
```

Notice how the timing climbs as each leading digit is fixed: locking in 7, then 3, then 7, then 3 steadily increases the measured time until the full code 7373 verifies successfully and unlocks Evil Harp mode.

**Flag: `flag{t1m1ng_4ttack}`**

## 6.5 Flag summary

| # | Challenge       | Flag                   |
|---|-----------------|------------------------|
| 1 | Static Analysis | flag{strings_4re_fr33} |
| 2 | USB Enumeration | flag{lu5b_v_1337}      |
| 3 | The Harp's Song | flag{MELODY}           |
| 4 | Side Channel    | flag{t1m1ng_4ttack}    |

---

*Have fun, and happy hacking.*